

Supervised, Unsupervised and Reinforcement Machine Learning Approach

1. Supervised Learning

Explanation

Supervised learning is learning with an answer key. You give the model thousands of examples where the correct answer (the **label**) is already known, such as emails marked "spam" or "not spam", and it learns to predict the label for new examples.

Types of supervised learning

- **Classification** predicts a category. Examples: spam or not spam, fraud or legitimate, which digit (0 to 9) is in an image.
- **Regression** predicts a continuous number. Examples: house price, delivery time, next month's sales.

Workflow

1. **Collect labelled data:** inputs with their known correct outputs.
2. **Prepare features:** convert inputs into numbers the model can use and clean missing values.
3. **Split the data:** typically 80% for training and 20% for testing.
4. **Train:** the model predicts, measures its error against the labels (the loss), and adjusts its parameters to reduce that error.
5. **Evaluate:** measure accuracy or error on the test data the model has never seen.
6. **Predict:** use the trained model on new, unlabeled inputs.

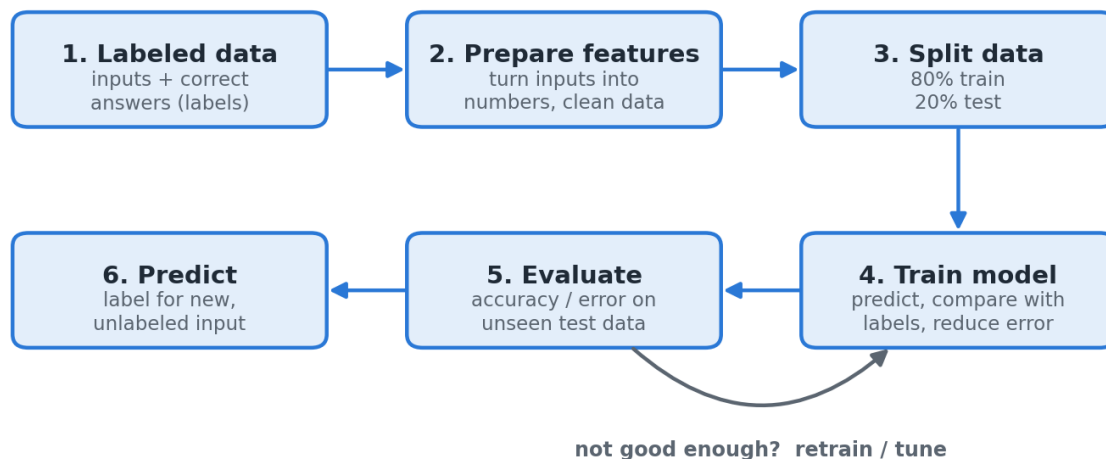


Figure 1: Supervised learning workflow

Common algorithms

Algorithm	Type	How it works
Linear Regression	Regression	Fits a straight line (or plane) between features and a numeric output.
Logistic Regression	Classification	Estimates the probability of a class, usually yes/no.
k-Nearest Neighbors (k-NN)	Both	Predicts from the labels of the k most similar training examples.
Decision Tree	Both	Learns a set of if/else rules that split the data.
Random Forest	Both	Combines many decision trees and takes a vote or average.

Real-world use cases

- Email spam and phishing filters
- Credit card fraud detection
- Medical image diagnosis (e.g. X-ray screening)
- Handwriting and postcode recognition
- House price and sales forecasting

Limitations of Supervised Learning

- Performance depends heavily on the **quality and accuracy of the labels**.
- May perform poorly when the new data is very different from the training data.
- Requires a large amount of **labelled data**, which can be expensive and time-consuming to create.

2. Unsupervised Learning

Explanation

Unsupervised learning is learning without an answer key. The model gets raw data and has to find structure on its own, for example grouping customers with similar buying habits, even though nobody told it what the groups should be.

Types of unsupervised learning

- **Clustering** groups similar data points together (e.g. customer segments).
- **Anomaly detection** finds data points that do not fit the normal pattern.
- **Association rules** find items that frequently occur together.

Workflow

1. **Collect raw data:** no labels are required.
2. **Select and scale features** so no feature dominates because of its units.
3. **Run the algorithm** to find groups, patterns or outliers.
4. **Interpret the result:** the algorithm returns group numbers; a person inspects and names each group.
5. **Act on it:** targeted marketing, investigation of anomalies, or input to other models.

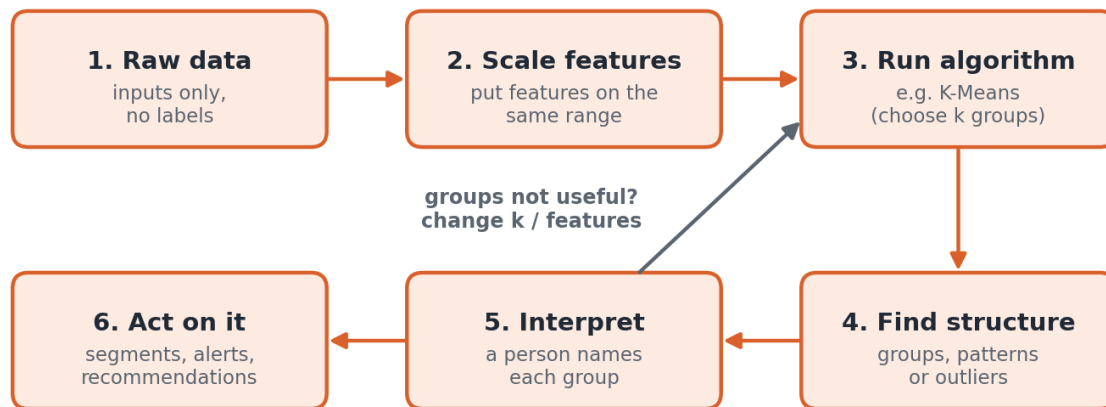


Figure 2: Unsupervised learning workflow

Common algorithms

Algorithm	Type	How it works
K-Means	Clustering	Splits data into k groups around centre points.
Hierarchical Clustering	Clustering	Builds a tree of groups within groups.
DBSCAN	Clustering	Finds dense groups of any shape and marks noise points.
Isolation Forest	Anomaly detection	Flags points that are easy to isolate from the rest.
Apriori	Association rules	Finds item combinations that often appear together.

Real-world use cases

- Customer segmentation for marketing
- Fraud and network intrusion detection (anomalies)
- Product and content recommendations ("people also viewed")
- Grouping research articles into topics
- Detecting clusters of suspiciously similar manuscripts (paper mills)

Limitations of Unsupervised Learning

- Results can be **difficult to interpret or validate**, because there is no predefined correct answer.
- The quality of the results can depend strongly on **feature selection and scaling**.
- Some algorithms require assumptions about the shape, size, or density of clusters.
- The number of clusters may need to be chosen in advance for methods such as K-Means.
- Detected patterns or anomalies may not always have a useful real-world meaning.

3. Reinforcement Learning

Simple Explanation

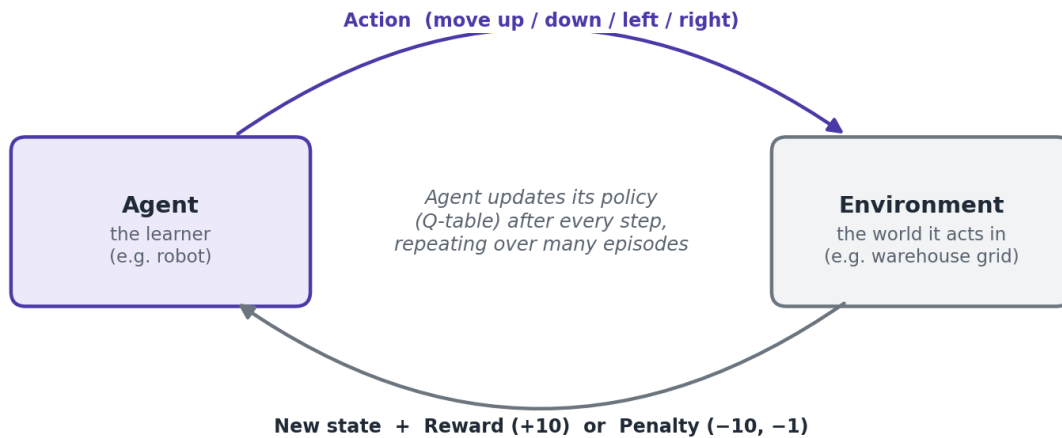
Reinforcement learning (RL) is learning by trial and error, like training a dog with treats. An **agent** takes actions in an **environment**, receives a **reward** (or penalty), and gradually learns which actions lead to the most reward over time.

Rewards and penalties

- **Rewards** encourage desired behaviour, e.g. +10 for reaching the goal.
- **Penalties** discourage undesired behaviour, e.g. -10 for falling into a hole.
- **Small step penalties** (e.g. -1 per move) push the agent toward the shortest route.
- The agent optimizes **cumulative** reward, so it can accept a small penalty now for a larger reward later.

Workflow

1. The agent observes its current state.
2. It chooses an action: sometimes random (exploration), usually the best-known action (exploitation).
3. The environment returns the next state and a reward or penalty.
4. The agent updates its estimate of how good that action was.
5. Repeat over many episodes until the policy stops improving.



Goal: learn the policy that maximizes total reward over time

Figure 3: Reinforcement learning loop between agent and environment

Common algorithms

Algorithm	How it works
Q-Learning	Learns a table of values for every state and action.
Deep Q-Network (DQN)	Replaces the table with a neural network for very large state spaces (e.g. game screens).
Policy Gradient / PPO	Learns the policy directly; used in robotics and to fine-tune chat models (RLHF).
Multi-Armed Bandits	Simplest form: repeatedly choose between options to find the best-paying one.

Real-world use cases

- Game-playing AI (AlphaGo, Atari games)
- Robotics and warehouse automation
- Self-driving car decision-making
- Traffic-signal control
- Fine-tuning AI assistants with human feedback (RLHF)
- Recommendation systems that learn from clicks over time

Limitations of Reinforcement Learning

- Training can be **slow** because the agent may need many episodes to learn a good policy.
- Designing suitable **rewards and penalties** can be difficult and can strongly affect what the agent learns.

4. Practical Demo

- Check out this example to understand how these approaches work [Machine Learning Approaches](#).

5. Key Terminology

Term	Meaning	Applies to
Training data	The examples a model learns from.	All
Features	Measurable inputs the model uses (e.g. pixel values, income).	Supervised, Unsupervised
Labels	The correct answer attached to each training example.	Supervised
Model	The learned function that maps inputs to outputs.	All
Training	Adjusting the model's parameters so it performs better.	All
Test data	Data held back from training to check performance on unseen examples.	Supervised
Loss function	A measure of how wrong the model's predictions are.	Supervised
Overfitting	When a model memorises training data and performs poorly on new data.	Supervised
Prediction (inference)	Using a trained model on new input.	All
Cluster	A group of similar data points found by the algorithm.	Unsupervised
Centroid	The centre point (average) of a cluster.	Unsupervised
Agent	The learner and decision-maker.	Reinforcement
Environment	Everything the agent interacts with; returns states and rewards.	Reinforcement
State	The agent's current situation.	Reinforcement
Actions	The choices the agent can make in a state.	Reinforcement
Rewards	Positive feedback for good outcomes.	Reinforcement

Term	Meaning	Applies to
Penalties	Negative feedback (negative rewards) for bad outcomes.	Reinforcement
Policy	The agent's strategy: which action to take in each state.	Reinforcement
Episode	One complete attempt from start to finish.	Reinforcement
Exploration vs exploitation	Trying new actions to learn versus using the best-known action.	Reinforcement

6. Comparison of Supervised, Unsupervised and Reinforcement Learning

Aspect	Supervised	Unsupervised	Reinforcement
Data	Labeled (input + correct output)	Unlabeled (input only)	No fixed dataset; generated by interaction
Feedback	Correct answer for every example	None	Reward or penalty after actions
Goal	Predict labels for new data	Discover hidden patterns or groups	Learn a policy that maximizes cumulative reward
Output	Category or number	Clusters, reduced features, anomaly scores	Best action for each state
Key algorithms	Linear/Logistic Regression, k-NN, Decision Tree, Random Forest, SVM	K-Means, DBSCAN, Hierarchical, PCA	Q-Learning, SARSA, DQN, PPO
Data cost	High: labeling takes effort	Low: raw data is enough	Needs a simulator or safe environment
Main challenge	Getting enough good labels; overfitting	Results can be hard to interpret or validate	Slow training; reward design is tricky
Example	Spam filter	Customer segmentation	Game-playing AI

7. When to Use Which Approach and Algorithm

Step 1: choose the learning approach

Ask yourself	If yes, use
Do I have historical data with the correct answer (label) for each example, and do I want to predict that answer for new data?	Supervised Learning
Do I have data but no labels, and want to find groups, patterns or unusual items?	Unsupervised Learning
Is there an agent that must make a sequence of decisions and can learn from rewards and penalties (often in a simulator)?	Reinforcement Learning

Step 2 (Supervised): what are you predicting?

- **A category** (yes/no, spam, digit 0-9) → Classification algorithm.
- **A number** (price, time, quantity) → Regression algorithm.

Step 3: choose the algorithm

Supervised algorithms

Algorithm	Use it when	Avoid it when
Linear Regression	Predicting a number and the relationship is roughly a straight line; you need a simple, explainable model.	The relationship is strongly non-linear.
Logistic Regression	Yes/no classification where you want probabilities and an explainable baseline.	Classes cannot be separated by a simple boundary.
k-Nearest Neighbors	Small to medium datasets where similar inputs have similar outputs (e.g. digit images).	Very large datasets (slow at prediction time) or many irrelevant features.
Decision Tree	You need rules that people can read and explain; mixed numeric and categorical data.	You need top accuracy; a single tree overfits easily.
Random Forest	Tabular business data where you want strong accuracy with little tuning.	You need a fully explainable model or very fast predictions.

Unsupervised algorithms

Algorithm	Use it when	Avoid it when
K-Means	You roughly know how many groups to expect and groups are round and similar in size (e.g. customer segments).	Groups have odd shapes or very different sizes; lots of outliers.

Algorithm	Use it when	Avoid it when
Hierarchical Clustering	You want to see groups within groups (a tree) or don't know how many groups there are; small datasets.	Datasets are large (it becomes slow).
DBSCAN	Groups have irregular shapes, the number of groups is unknown, and you want outliers flagged as noise.	Groups have very different densities.

Reinforcement learning algorithms

Algorithm	Use it when	Avoid it when
Q-Learning	The number of states and actions is small enough to fit in a table (e.g. grid worlds, simple games).	States are huge or continuous (e.g. images, sensor readings).
SARSA	Mistakes during learning are costly and you want a safer, more cautious policy.	You want the most aggressive optimal policy.
Deep Q-Network (DQN)	States are too many for a table (e.g. game screens) and actions are discrete.	Actions are continuous (e.g. steering angle).
Policy Gradient / PPO	Actions are continuous (robot joints, steering) or the problem is complex; fine-tuning language models.	A simple table-based method would already work.
Multi-Armed Bandits	Repeatedly choosing between a few options with no long-term state (e.g. A/B testing headlines).	Actions affect future situations (use full RL instead).